

PYTHON FOR YOUNG INNOVATORS

From First Program to Your First AI Project

A complete 11-week Python and Machine Learning course for secondary school students, from your very first line of code to a finished, working ML project.

11

WEEKS

44

HOURS

0

EXPERIENCE NEEDED

1

FINAL ML PROJECT

CODE



DATA



MODEL



EXPLAIN

FROM `print("HELLO, WORLD!")` TO YOUR OWN MACHINE LEARNING MODEL

No prior coding or maths experience required

By Mr. Muwatta

Founder, Algorise Tech Explorers

muwatta.com.ng/academy

A Note From Mr. Muwatta

WELCOME

A Note From Mr. Muwatta

Welcome to this course. My name is Mr. Muwatta, founder of Algorise Tech Explorers. I built this course because I believe every student, whatever their background, can learn to code and go on to build real Artificial Intelligence and Machine Learning projects, as long as the material is explained clearly, in order, with enough practice at every step.

You do not need to be a "maths genius" or already know what programming looks like. You only need to be willing to type code, make mistakes, read what the computer tells you, and try again. That is exactly how real programmers and AI engineers work every day.

By the end of Week 11, you will have built your own small Machine Learning project, from choosing a problem to making a real prediction and explaining it to someone else. I am proud to guide you there.

VISIT ALGORISE ACADEMY ONLINE

This course now has an online home at muwatta.com.ng/academy, where you can follow these same lessons, track your progress, and try your code interactively. Use this workbook alongside the Academy, or on its own.

Mr. Muwatta

Founder, Algorise Tech Explorers
www.muwatta.com.ng

Table of Contents

CONTENTS

Table of Contents

PART 1 Course Overview

- Setup Guide (do this once, before Week 1)

PART 2 11-Week Curriculum Map

PART 3 Complete Student Course Material

- Week 1, Starting to Code
- Week 2, Making Decisions
- Weeks 3 to 11, continuing the course
- Phase 3, Python for Data (Weeks 6 to 7)
- Phase 4, Introduction to AI and Machine Learning (Week 8)
- Phase 5, Building AI/ML Projects (Weeks 9 to 10)
- Phase 6, Independent Final Project (Week 11)

PART 4 Exercises and Solutions

PART 5 Projects

- Weekly mini-projects, final project framework and options

PART 6 Assessments

- Quizzes, coding challenges, mid-course test, final rubric

PART 7 Final Project Readiness Matrix

Course Overview

PART 1

Course Overview

Field	Detail
Course Title	Python for Young Innovators: From First Program to Your First AI Project
Target Students	Basic 4, Basic 5, JSS1 to JSS3 (ages 9 to 15), beginners welcome
Duration	11 weeks
Total Hours	44 hours (2 sessions/week x 2 hours x 11 weeks)
Prerequisites	None. No prior programming, maths, or command-line experience needed. A laptop or desktop able to run Python is required.

Learning Philosophy

This course is practical first, incremental, and project driven. Students write code in nearly every session from Week 1. Every concept is explained from zero. The course is designed backward from one test: could a student finish the final project using only this material, even working alone? Every skill the final project needs is taught and practiced well before it is needed. Help is heaviest at the start and is deliberately removed week by week, so that by Weeks 10 and 11 students are making and defending their own project decisions.

Final Learning Outcomes

By the end of Week 11, every student should be able to:

- ▶ Write, run, and debug Python programs confidently
- ▶ Explain what a variable, loop, function, and error message are, in their own words
- ▶ Work with lists, dictionaries, and simple text files of data
- ▶ Load a small dataset with Pandas and describe what its rows and columns represent
- ▶ Explain the difference between a feature and a target
- ▶ Explain in plain language what "training a model" means
- ▶ Use scikit-learn to train a simple classification or regression model
- ▶ Test a model and describe how good or bad its predictions are
- ▶ Use a trained model to make a new prediction and interpret the result
- ▶ Modify an existing project and explain what changed and why
- ▶ Explain their finished project to another person without reading from a script

Course Overview

Your Path to Becoming an AI/ML Engineer

Everything in this workbook aims at one destination: real, honest first steps toward becoming an Artificial Intelligence and Machine Learning engineer. That may sound like a distant title, and that is normal. An AI/ML engineer is simply a person who has learned to do a small number of things well, and this course teaches every one of them, at a beginner's scale, in order.

An AI/ML engineer's daily work usually involves five things:

- ▶ Understanding a real problem. Someone needs a question answered, will this student pass, is this photo a cat or a dog, will this machine break down soon, and the engineer must understand the problem before touching any code
- ▶ Collecting and cleaning data. Real data is messy, with missing pieces and strange values. Engineers spend enormous amounts of time making data usable
- ▶ Choosing and training a model. Once data is ready, the engineer picks a method that can learn patterns from it and trains that method on real examples
- ▶ Testing whether it actually works. A model that looks good on paper can still be wrong. Engineers test carefully, on data the model has never seen, before trusting it
- ▶ Explaining the result to other people. A brilliant model nobody understands is not very useful. Engineers must explain, in plain language, what their system does and does not do well

By the end of Week 11, you will have personally done all five of these yourself, on your own small project. That is not a simplified imitation of the job, it is the same loop, at a smaller and gentler scale. Every "Career Corner" box in this course connects your practice directly back to this real profession, so that harder material later in life feels familiar rather than frightening.

Course Overview

Tools and Software Required

- ▶ A laptop or desktop computer, no special hardware needed
- ▶ Python 3 (installation covered in the Setup Guide below)
- ▶ A simple code editor: Thonny for beginners (built in, no configuration), or VS Code for students who want something closer to professional tools
- ▶ Internet access is helpful but not required for every lesson, installation happens once, and after that almost everything runs offline
- ▶ Libraries used, introduced gradually: pandas, numpy, matplotlib, scikit-learn (all free and open source)

CAREER CORNER · ON THE JOB AS AN AI/ML ENGINEER

Every AI and Machine Learning engineer working today began exactly where you are: writing tiny, simple lines of code and learning to think in exact, patient steps. Before anyone builds a self-driving car's vision system or a chatbot that talks to millions of people, they spend months becoming comfortable with the basics you are about to learn. There is no shortcut past this stage, and every professional you admire once sat exactly where you are sitting now.

Setup Guide (do this once, before Week 1)

1. What is Python?

Python is a programming language: a way of writing instructions that a computer can follow exactly, one step at a time. We use it because it reads almost like plain English and is one of the most widely used languages for AI and Machine Learning in the real world.

2. How to install Python

- ▶ Go to python.org/downloads (do this with an adult, or with me, if unsure)
- ▶ Download the version for your operating system (Windows or Mac)
- ▶ Run the installer. Important (Windows only): on the first screen, tick "Add Python to PATH" before clicking Install
- ▶ Finish the installation

3. How to check whether Python is installed

Open a terminal (Windows: search "Command Prompt"; Mac: search "Terminal") and type:

```
1 | python --version
```

If that does not work, try:

Course Overview

```
1 | python3 --version
```

You should see something like Python 3.12.1. An error usually means Python was not added to PATH during install, ask an adult or your instructor for help.

Course Overview

4. Install Thonny (recommended editor for this course)

- ▶ Go to thonny.org, download the version for your system, and install it like any program
- ▶ Open Thonny. You will see a white script area at the top and a "Shell" area at the bottom, that is where output and errors appear

5. How to create a project folder

- ▶ Create a folder called python-course somewhere easy to find, such as your Desktop
- ▶ Inside it, create a new folder for each week, e.g. week1, week2

6. How to create a Python file

In Thonny: File → New. Save it inside your week's folder with a name ending in .py, for example lesson1.py.

7. How to run Python code

Click the green Run button (or press F5). The output appears in the Shell area below.

8. How to install libraries

From Week 6 you will need extra libraries. In Thonny: Tools → Manage Packages, search for the library name (e.g. pandas), and click Install. If using a terminal instead:

```
1 pip install pandas numpy matplotlib scikit-learn
```

If pip is not recognized, try pip3 instead.

9. What to do if installation fails

- ▶ Restart your computer and try again
- ▶ Make sure you have internet access for this one-time step
- ▶ Ask your instructor to install it together in class, this is normal

11-Week Curriculum Map

PART 2

11-Week Curriculum Map

We ek	Main Topic	Python Skills	Practical Project
1	Starting to Code	print, comments, variables, data types, input	"About Me" and "Mini Calculator" programs
2	Making Decisions	operators, comparisons, if/elif/else, logical operators	"Pass or Fail" and "Weather Advisor" programs
3	Repeating & Organizing Data	for loops, while loops, lists, indexing	"Class Score Tracker"
4	Reusable Code & Dictionaries	functions, parameters, return values, dictionaries	"Student Record" program
5	Files & Errors	reading/writing text files, basic exception handling	"Save and Load Scores" program
6	Meet Your Data	datasets, rows/columns, features/labels, reading CSV with Pandas	Load and explore a small CSV
7	Exploring Data	Pandas and NumPy basics, simple Matplotlib charts	Chart from real data (e.g. scores per subject)
8	What Is AI & ML, Really?	no new syntax, concept week	"Human as a Model" unplugged activity
9	Your First Model	scikit-learn basics: train/test split, .fit(), .predict(), accuracy	Guided classification project
10	Build With Less Help	choosing features/target, evaluating and experimenting	Guided-but-lighter project of choice
11	Your Own Project	independent build, debugging, presenting	Final Project (Part 5)

Every week also names its AI/ML Connection and Expected Outcome in Part 3, alongside the full lesson material.

Complete Student Course Material

PART 3

Complete Student Course Material

Each week has two sessions. Every session follows the same shape: warm-up, explanation, guided coding, practice, challenge, wrap-up. If you missed a class, start from "What You Will Learn" and work through in order, everything you need is explained here.

WEEK 1

1. Starting to Code

WEEKLY GOAL

By the end of this week, you will write a Python program that stores information in variables, prints messages, and asks the user questions, completely from scratch.

CAREER CORNER · ON THE JOB AS AN AI/ML ENGINEER

Every AI and Machine Learning engineer working today began exactly where you are: writing tiny, simple lines of code and learning to think in exact, patient steps. Before anyone builds a self-driving car's vision system or a chatbot used by millions, they spend months becoming comfortable with the basics you are learning this week: storing information, giving clear instructions, and reading error messages without panicking.

Session 1: What Is Programming? Your First Lines of Python

What You Will Learn

- ▶ Explain what a computer program is
- ▶ Explain what Python is and why we use it
- ▶ Write and run a program using `print()`
- ▶ Add comments to explain your code

Why Does This Matter?

A computer cannot guess what you want, it only does exactly what you tell it, in an order it understands. Programming is the skill of giving a computer clear, exact instructions. Every AI system, however advanced, is built from ordinary instructions like the ones you will write today.

Complete Student Course Material

DID YOU KNOW?

Python is the single most widely used programming language among professional AI and Machine Learning engineers. It was not originally designed for AI, it was created in 1991 to be easy to read and teach. That very quality, being easy for humans to read, is why it later became the favorite language for building intelligent systems.

Key Words

- ▶ Program: a set of instructions that tells a computer what to do
- ▶ Python: the programming language we use in this course
- ▶ Code: the instructions you write, in a language the computer understands
- ▶ Run: to make the computer follow your instructions
- ▶ Comment: a note in your code that Python ignores, for humans to read

Learn the Concept

A computer does what you tell it, step by step, top to bottom. One of the simplest instructions in Python is `print()`. It tells the computer to show text to the person using the program.

```
1 print("Hello, World!")
```

`print` is a function, a built-in command that does something (you will build your own functions in Week 4). The round brackets hold what it acts on. Text inside quotation marks is called a string, Python's word for a piece of text.

You can also leave notes for yourself using `#`. Python ignores anything after a `#` on that line:

```
1 # This program greets the world
2 print("Hello, World!") # this line displays a message
```

First Example

```
1 # My first Python program
2 print("Hello, World!")
3 print("I am learning Python.")
```

Line 1 is a comment, Python skips it. Line 2 displays the text. Line 3 displays a second message on a new line. Run it, you should see:

```
Hello, World!
I am learning Python.
```

Complete Student Course Material

Try It Yourself

Write a program that prints your name, your school, and one thing you hope to learn in this course, as three separate `print()` lines.

Hint. You need three separate `print()` lines, one under the other. Each piece of text goes inside quotation marks.

Check Yourself

1. What symbol does Python use to start a comment?
2. What will `print("2 + 2")` print? Read it carefully.
3. Why does the text in `print()` need quotation marks?

```
1 print("Hello, World!")
```

DEBUGGING CHALLENGE

What is wrong with this program? Hint: count the brackets. Try running it yourself first and read what Python tells you before checking the fix.

MINI CHALLENGE

Problem: write a short "About Me" program. Requirements: at least 4 `print()` lines, including your name, age, favorite subject, and one comment explaining what the program does.

Hint. Start with a comment line, then your four `print()` statements. Optional harder version: add a blank `print()` line between two lines to create spacing.

AI / ML CONNECTION

Every AI system starts the same way this program did: with instructions a computer can follow exactly. Later, programs will look at data and decide what to print based on patterns, but the starting skill is the same one you just used: telling the computer, precisely, what to do.

Practice Tasks

- ▶ Easy: print your favorite food and favorite color, one per line
- ▶ Easy: print a 3-line poem, it does not need to rhyme
- ▶ Medium: print a small menu for a fictional shop, 3 items with prices, one per line
- ▶ Medium: add a comment above each `print()` line in your menu explaining what it does
- ▶ Challenge: use `print()` to draw a simple shape with text characters, e.g. a small triangle of asterisks

Complete Student Course Material

Exit Ticket

1. What does print() do?
2. What is a comment used for?
3. What would happen if you forgot a closing quotation mark?
4. Name one thing you found confusing today.

What You Should Be Able to Do Now

- ☐ I can open Thonny and create a new Python file
- ☐ I can use print() to display text
- ☐ I can write comments in my code
- ☐ I can run my program and see the output

Complete Student Course Material

Session 2: Variables and Data Types

What You Will Learn

- ▶ Create a variable and store a value in it
- ▶ Change a variable's value
- ▶ Use a variable inside print()
- ▶ Identify string, integer, float, and boolean values
- ▶ Use input() to get information from the user

Why Does This Matter?

So far we only printed fixed text. Real programs, and every AI system, need to store information so they can use it later. A variable stores a single piece of information under a name we choose. Machine learning is, at its core, finding patterns in huge collections of stored values, today you learn the smallest unit of that: one variable holding one value.

Key Words

- ▶ Variable: a name used to store a value
- ▶ Value: the actual information stored
- ▶ String: text data, written in quotes, e.g. "Ada"
- ▶ Integer (int): a whole number, e.g. 12
- ▶ Float: a decimal number, e.g. 12.5
- ▶ Boolean (bool): either True or False
- ▶ Input: information typed in by the person using the program

Learn the Concept

```
1 name = "Abdullahi"
```

name is the variable, a label we chose. "Abdullahi" is the value stored inside it. The = sign does not mean "equals" the way it does in maths, it means store this value in this name. Variables can hold different types of data:

```
1 name = "Abdullahi" # string (text)
2 age = 12 # integer (whole number)
3 height = 1.45 # float (decimal number)
4 is_student = True # boolean (True or False)
```

You can print a variable directly, or combine text and variables separated by commas:

```
1 print(name)
2 print("My name is", name, "and I am", age, "years old.")
```

Complete Student Course Material

You can also ask the user to type something in with `input()`. Whatever the user types is always stored as a string, even if it looks like a number:

```
1 name = input("What is your name? ")  
2 print("Hello,", name)
```

Complete Student Course Material

First Example

```
1 name = "Abdullahi"
2 age = 12
3 print("My name is", name)
4 print("I am", age, "years old")
5 age = 13
6 print("Next year I will be", age)
```

Lines 1 to 2 create two variables. Line 3 prints text combined with name. Line 4 prints text combined with age. Line 5 changes the value stored in age, the old value is replaced. Output:

```
My name is Abdullahi
I am 12 years old
Next year I will be 13
```

Try It Yourself

Write a program that creates variables for your name, age, and favorite subject, prints a sentence using all three in one `print()` line, then uses `input()` to ask the user for their own name and greets them.

Hint. You need one variable per piece of information, and you can list several things inside one `print()` separated by commas.

Check Yourself

1. What will this print? `score = 70; print(score); score = 85; print(score)`
2. What data type is "12", is it the same as 12? Why might this matter?
3. If you use `input()` to ask someone's age, what type will Python store it as?

```
1 Name = "Zainab"
2 print(name)
```

DEBUGGING CHALLENGE

What is wrong with this program? Hint: Python is case sensitive, look closely at both lines.

MINI CHALLENGE

Problem: build a small "Student Profile" program. Requirements: ask the user for their name, age, and favorite subject using `input()`, store each answer in a variable, then print a full sentence introducing them using all three.

Hint. You need three `input()` lines and one combined `print()` line at the end. Optional harder version: also ask for a score using `input()`, and print it back with the word "points" after it.

Complete Student Course Material

AI / ML CONNECTION

Think of a spreadsheet of exam scores: each student's name, age, and score are stored somewhere. A variable is the smallest possible version of that idea, one labeled box holding one piece of information. Later we will use lists and dictionaries to store many of these at once (Weeks 3 to 4), and eventually whole datasets with thousands of rows (Week 6).

Practice Tasks

- ▶ Easy: create three variables (name, favorite animal, favorite number) and print them in one sentence
- ▶ Easy: ask the user for their age using `input()` and print "You are ____ years old"
- ▶ Medium: create variables for a book's title, author, and page count, and print a short book card
- ▶ Medium: ask the user for two numbers using `input()`, and print both back labeled clearly
- ▶ Challenge: create a "Fact File" about an animal using at least one string, one integer, one float, and one boolean variable, and print all of them

Exit Ticket

1. What is the difference between a variable's name and its value?
2. Name the four data types you learned today.
3. What type does `input()` always give you?
4. What would happen if you tried to use a variable before creating it?

What You Should Be Able to Do Now

- ☐ I can create a variable
- ☐ I can store text, whole numbers, decimals, and True/False in variables
- ☐ I can print a variable and combine variables with text in one `print()`
- ☐ I can change a variable's value
- ☐ I can use `input()` to collect information from the user

Complete Student Course Material

WEEK 2

2. Making Decisions

WEEKLY GOAL

By the end of this week, you will write programs that compare values and make different decisions depending on the situation.

CAREER CORNER · ON THE JOB AS AN AI/ML ENGINEER

AI and Machine Learning engineers spend a large part of their day writing conditions: deciding what a program should do when data looks one way versus another. When you later build a rule such as "if the model's confidence is above 80 percent, trust its prediction, otherwise ask a human to check it," you will be using exactly the if/elif/else skills you practice this week. Decision-making code like this sits underneath almost every real AI system, making choices thousands of times per second.

Session 1: Operators and Comparisons

What You Will Learn

- ▶ Use arithmetic operators (+ - * / // %) to calculate values
- ▶ Convert between data types using int(), float(), str()
- ▶ Compare two values using == != > < >= <=
- ▶ Understand that comparisons produce a boolean (True/False)

Why Does This Matter?

Decisions in programming, and in AI, are built entirely from comparisons: is this score higher than 50, is this temperature above freezing. Every one of those questions becomes a boolean. Before we can make programs decide anything, we need to be fluent in asking exact, comparable questions about our data.

Key Words

- ▶ Operator: a symbol that performs an action on values, e.g. +
- ▶ Type conversion: changing a value from one data type to another
- ▶ Comparison operator: a symbol that compares two values and returns True or False

Learn the Concept

Complete Student Course Material

```
1 print(5 + 3) # addition -> 8
2 print(5 - 3) # subtraction -> 2
3 print(5 * 3) # multiplication -> 15
4 print(5 / 3) # division -> 1.666...
5 print(5 // 3) # whole-number division -> 1
6 print(5 % 3) # remainder ("modulo") -> 2
```

Recall that `input()` always gives text. To do maths with a typed answer, convert it:

```
1 age_text = input("How old are you? ")
2 age = int(age_text) # convert string to integer
3 print(age + 1)
```

You can also convert the other way, e.g. `str(12)` turns the number 12 into the text "12", useful when combining numbers into a sentence with `+` instead of a comma.

Comparison operators ask a yes or no question about two values, and the answer is always a boolean:

```
1 score = 75
2 print(score > 50) # True
3 print(score == 100) # False
4 print(score != 100) # True
```

Note: `==` (double equals) checks if two values are equal. A single `=` stores a value, they are not the same thing, and mixing them up is one of the most common beginner mistakes.

Complete Student Course Material

First Example

```
1 score = 68
2 print(score > 50)
3 print(score >= 68)
4 print(score == 70)
```

Line 1 stores 68 in score. Line 2 asks "is 68 greater than 50?", True. Line 3 asks "is 68 greater than or equal to 68?", True. Line 4 asks "does 68 equal 70?", False.

Try It Yourself

Ask the user for two numbers using `input()`, convert both to integers, then print whether the first is greater than the second.

Hint. You need `int(input(...))` twice, then one comparison inside a `print()`.

Check Yourself

1. What is $17 \% 5$? Work it out before running it.
2. What is the difference between `=` and `==`?
3. Why does `input()` need `int()` before we can do maths with it?

```
1 age = input("Enter your age: ")
2 next_year = age + 1
3 print(next_year)
```

DEBUGGING CHALLENGE

What is wrong, and why does Python complain about combining a string and an integer?

MINI CHALLENGE

Problem: build a small "Number Checker." Requirements: ask the user for a number, convert it to an integer, then print three results: whether it is greater than 10, whether it is exactly 0, and its remainder when divided by 2.

Hint. You need one `input()`, one `int()` conversion, and three `print()` lines using comparison or arithmetic operators.

AI / ML CONNECTION

Machine learning models constantly ask comparison-style questions about data: "is this measurement bigger than that threshold?" The comparisons you just wrote by hand are, in a simplified way, exactly the kind of question a trained model asks automatically, thousands of times, once it has learned from data.

Complete Student Course Material

Practice Tasks

- ▶ Easy: print the result of $9 * 9$
- ▶ Easy: ask for a number and print whether it is even (hint: use %)
- ▶ Medium: ask for two numbers and print whether they are equal
- ▶ Medium: ask for a height in centimeters and print whether it is above 150
- ▶ Challenge: ask for a temperature in Celsius, print whether it is above, exactly, or below 0, as three separate comparisons

Exit Ticket

1. What symbol checks if two values are equal?
2. What does // do differently from /?
3. Why did the debugging challenge program crash?
4. What data type does a comparison always produce?

What You Should Be Able to Do Now

- ☐ I can use + - * / // % to calculate values
- ☐ I can convert between string, int, and float using int(), float(), str()
- ☐ I can compare two values using == != > < >= <=
- ☐ I understand that comparisons produce True or False

Complete Student Course Material

Session 2: if, elif, else

What You Will Learn

- ▶ Use if to run code only when a condition is true
- ▶ Use else to handle the opposite case
- ▶ Use elif to check multiple conditions in order
- ▶ Combine conditions using and, or, not

Why Does This Matter?

This is the moment your programs start branching: doing different things in different situations, instead of running the same steps every time. This is the biggest building block between "a program that just displays things" and "a program that behaves intelligently." Every classification decision an ML model makes (pass/fail, cat/dog, spam/not spam) is, underneath, built from this same idea repeated automatically.

Key Words

- ▶ Condition: a statement that is either True or False
- ▶ if statement: runs a block of code only if a condition is True
- ▶ else: runs a block of code when the if condition is False
- ▶ elif: short for "else if", checks another condition if the first was False
- ▶ Logical operator: and, or, not, combine or reverse conditions

Learn the Concept

```
1 score = 45
2 if score >= 50:
3     print("You passed!")
4 else:
5     print("You need to try again.")
```

Notice the indentation before print, Python uses this to know which lines belong inside the if. This is part of the language's grammar, not optional. To check several conditions in order, use elif:

```
1 score = 78
2 if score >= 80:
3     print("Grade: A")
4 elif score >= 60:
5     print("Grade: B")
6 elif score >= 40:
7     print("Grade: C")
8 else:
9     print("Grade: F")
```

Complete Student Course Material

Python checks each condition top to bottom and stops at the first True one. You can combine conditions:

```
1 age = 13
2 has_permission = True
3 if age >= 12 and has_permission:
4     print("You may join the trip.")
```

and requires both sides to be True. or requires at least one side to be True. not flips True to False and vice versa.

Complete Student Course Material

First Example

```
1 temperature = 15
2 if temperature > 30:
3     print("It's hot outside.")
4 elif temperature > 15:
5     print("It's warm outside.")
6 else:
7     print("It's cool outside.")
```

Python checks `temperature > 30`, false. It checks `temperature > 15`, false (15 equals 15, not greater). None matched, so it runs the else block: "It's cool outside."

Try It Yourself

Write a program that asks the user for their exam score (0 to 100) and prints their grade using at least three elif branches.

Hint. Start from the highest grade boundary and work downward, like the temperature example.

Check Yourself

1. What happens if none of the if/elif conditions are true and there is no else?
2. In the grade example, why must we check `score >= 80` before `score >= 60`?
3. What does "not True" evaluate to?

```
1 score = 55
2 if score >= 50
3     print("Pass")
4 else:
5     print("Fail")
```

DEBUGGING CHALLENGE

What is wrong with this program? Look carefully at the end of the if line.

MINI CHALLENGE

Problem: build a "Pass or Fail" program. Requirements: ask for a student's name and three scores, calculate the average, then print whether the student passed (average of 50 or above), including their name in the message.

Hint. Add the three scores and divide by 3 to get the average before your if statement. Optional harder version: add elif branches for "Distinction" (75 or above) and "Needs Improvement" (below 40).

Complete Student Course Material

AI / ML CONNECTION

A trained ML classifier is, in effect, an enormous, automatically learned version of an if/elif/else chain, except instead of a human writing thresholds like "score $\geq$ 50," the model learns the best thresholds itself from examples. Understanding how to write these rules by hand is exactly what makes Week 8's explanation of "what a model learns" make sense.

Practice Tasks

- ▶ Easy: ask for a number and print "positive," "negative," or "zero"
- ▶ Easy: ask for an age and print whether they can vote (18+)
- ▶ Medium: build a "Weather Advisor": ask for temperature and rain (yes/no), give clothing advice using and/or
- ▶ Medium: build a password checker: print whether a password is at least 8 characters long (hint: len(password))
- ▶ Challenge: build a ticket price calculator with at least 3 age brackets

Exit Ticket

1. What is the purpose of elif?
2. Why does indentation matter in Python?
3. What is the difference between and and or?
4. What is wrong with "if score = 50:"?

What You Should Be Able to Do Now

- ☐ I can write an if/else statement
- ☐ I can chain multiple conditions with elif
- ☐ I can combine conditions with and, or, not
- ☐ I understand why indentation matters in Python

Complete Student Course Material

Weeks 3 to 11: Continuing the Course

These weeks follow the same lesson shape as Weeks 1 and 2, written a little more compactly: objectives, a clear explanation with runnable code, a guided example, a try-it-yourself task with a hint, a debugging challenge, a mini-project, an AI/ML connection, and practice tasks, everything needed to work through the week alone.

Complete Student Course Material

WEEK 3

3. Repeating and Organizing Data (Loops and Lists)

WEEKLY GOAL

Store many values at once in a list, and use loops to process every item without repeating code by hand.

CAREER CORNER · ON THE JOB AS AN AI/ML ENGINEER

Real datasets used by AI and Machine Learning engineers can contain millions of rows, far too many to look at one by one. Engineers rarely process data by hand, instead they write a loop once and let the computer repeat it perfectly. The pattern you learn this week, store many values then process every one automatically, is the exact pattern used to scan a million photographs while training an image-recognition model.

Key Words

List, index, for loop, while loop, iteration.

Concept

```
1 scores = [70, 85, 92, 61, 78]
2 print(scores[0]) # first item -> 70
3 print(scores[-1]) # last item -> 78
4 scores.append(100) # add a new item
5 print(len(scores)) # how many items -> 6
```

Lists are numbered starting at 0, not 1, this trips up every beginner at least once.

```
1 scores = [70, 85, 92, 61, 78]
2 for score in scores:
3     print(score)
```

This runs the indented block once for every item in the list. Combine with if:

```
1 for score in scores:
2     if score >= 70:
3         print(score, "- Pass")
4     else:
5         print(score, "- Fail")
```

Complete Student Course Material

```

1 count = 0
2 while count < 5:
3     print("Count is", count)
4     count = count + 1

```

while repeats as long as a condition stays True. Forgetting to change the condition variable inside the loop causes an infinite loop, a common and easily fixable mistake (Ctrl+C, or the Stop button in Thonny, stops it).

Guided Example

```

1 scores = [70, 85, 92, 61, 78]
2 total = 0
3 for score in scores:
4     total = total + score
5 average = total / len(scores)
6 print("Average score:", average)

```

Try It Yourself

Given a list of 5 exam scores, print only the scores above the average.

Hint. Calculate the average first, then loop through the list again with an if.

```

1 scores = [70, 85, 92]
2 for score in scores
3     print(score)

```

DEBUGGING CHALLENGE

What is missing? Same family of mistake as Week 2's if statement.

MINI-PROJECT

Ask the user to enter 5 scores one at a time using a while loop, store each in a list, then print the total, the average, the highest score (hint: `max(scores)`), and the lowest (`min(scores)`).

AI / ML CONNECTION

A dataset is essentially a very long list, often thousands or millions of rows. A for loop is how a computer looks at every row, one at a time, which is exactly what happens behind the scenes when a model is trained or when we explore a dataset in Week 6 to 7.

Practice Tasks

- Easy: print each item in a list of your 5 favorite foods

Complete Student Course Material

- ▶ Easy: count how many numbers in a list are even
- ▶ Medium: find the largest number in a list without using `max()`
- ▶ Medium: reverse a list by printing it from last to first, using a loop and indexing
- ▶ Challenge: given a list of student names and a separate list of scores in the same order, print each name next to its score using an index

Complete Student Course Material

WEEK 4

4. Reusable Code and Dictionaries (Functions)

WEEKLY GOAL

Write your own reusable functions, and store labeled data using dictionaries.

CAREER CORNER · ON THE JOB AS AN AI/ML ENGINEER

Open the code of any working AI or Machine Learning engineer and you will see the same shape you learn this week appearing again and again: a function that takes some input and returns a decision or a number. That is, in miniature, exactly what a trained Machine Learning model does. Dictionaries matter too, a great deal of real-world data arrives labeled and organized exactly like a dictionary, long before it is tidied into a clean spreadsheet.

Key Words

Function, parameter, argument, return value, dictionary, key, value.

Concept

```
1 def greet(name):
2     print("Hello,", name)
3
4     greet("Amaka")
5     greet("Tunde")
```

def defines a new function. name is a parameter, a placeholder for whatever value is passed in when the function is called. "Amaka" is the argument, the actual value used that time. Functions can also return a value to be used later, instead of just printing:

```
1 def add(a, b):
2     return a + b
3
4     result = add(5, 3)
5     print(result)
```

A dictionary stores data as labeled key-value pairs, much closer to how a real record (one row of a dataset) looks:

```
1 student = {"name": "Zainab", "age": 13, "score": 88}
2 print(student["name"])
3 print(student["score"])
4 student["score"] = 91 # update a value
```

Complete Student Course Material

Guided Example

```
1 def describe_student(student):  
2     print(student["name"], "is", student["age"], "years old and scored",  
3         student["score"])  
4  
5 student1 = {"name": "Chinedu", "age": 12, "score": 76}  
6 describe_student(student1)
```

Try It Yourself

Write a function `is_passing(score)` that returns `True` if the score is 50 or above, and `False` otherwise. Test it with three different scores.

Hint. Use `return`, not `print`, inside the function, then use `print()` when you call it.

```
1 def add(a, b)  
2     return a + b  
3  
4 print(add(2, 3))
```

DEBUGGING CHALLENGE

Find the missing symbol, same family of mistake as loops and if.

MINI-PROJECT

Create a dictionary for each of 3 students (name, age, three scores as a list). Write a function `average_score(student)` that returns the average of a student's scores. Loop through all three students and print each name and average using your function.

AI / ML CONNECTION

A dictionary is close in shape to one row of a real dataset, a set of labeled features about one thing. A function that takes data in and gives a decision or number back out is, conceptually, exactly what a trained ML model is: something that takes features in and returns a prediction out.

Practice Tasks

- ▶ Easy: write a function `square(n)` that returns a number multiplied by itself
- ▶ Easy: create a dictionary describing your favorite book and print each value
- ▶ Medium: write a function `grade(score)` that returns "A", "B", "C" or "F"
- ▶ Medium: create a list of 3 student dictionaries and print each name and pass/fail using `is_passing()`

Complete Student Course Material

- ▶ Challenge: write a function `class_average(students)` that returns the average of all students' scores combined

Complete Student Course Material

WEEK 5

5. Files and Errors

WEEKLY GOAL

Save data to a file and load it back, and handle simple errors without panicking.

CAREER CORNER · ON THE JOB AS AN AI/ML ENGINEER

AI and Machine Learning engineers work with real files nearly every day: downloaded datasets, saved trained models, logs of what a system decided and why. Learning to open a file safely, read what is inside it, and calmly handle a missing or damaged file is a genuine, everyday part of the profession, and engineers who are careless about it lose data and waste hours.

Key Words

File, read, write, exception, try/except.

Concept

```
1 scores = [70, 85, 92]
2 file = open("scores.txt", "w")
3 for score in scores:
4     file.write(str(score) + "\n")
5 file.close()
```

"w" means write mode, this creates the file or replaces it if it exists. `str(score)` converts the number to text, since files store text. `"\n"` moves to a new line.

```
1 file = open("scores.txt", "r")
2 for line in file:
3     print(line.strip()) # .strip() removes the extra newline
4 file.close()
```

```
1 try:
2     number = int(input("Enter a number: "))
3     print(100 / number)
4 except ValueError:
5     print("That wasn't a valid number.")
6 except ZeroDivisionError:
7     print("You can't divide by zero.")
```

This lets your program respond calmly to a mistake instead of crashing, exactly what real data-handling code must do, since real-world data is often messy or incomplete.

Guided Example

Complete Student Course Material

Combine writing, reading, and error handling: save 3 scores typed by the user to a file, using try/except in case they type something that is not a number.

Try It Yourself

Write a program that reads scores.txt (created above) and prints the total of all the scores in it.

Hint. Every line read from a file is text, you will need int() on each line after .strip().

```
1 file = open("scores.txt", "r")
2 data = file.read()
3 print(data)
```

DEBUGGING CHALLENGE

What happens if scores.txt does not exist yet? Run it and read the error message, what type of error is it? (This previews FileNotFoundError.)

MINI-PROJECT

Build a program with two parts: ask the user to type 5 scores and save them to class_scores.txt, then separately read the file back, calculate and print the average, using try/except to handle a missing file with a friendly message instead of crashing.

AI / ML CONNECTION

Every dataset used in Machine Learning lives in a file, very often a .csv file, before it is loaded into a program. What you just did, writing data out and reading it back in, is the exact same skill, at a small scale, that Week 6 uses to load a real dataset with Pandas.

Practice Tasks

- ▶ Easy: write your name and school to a file called me.txt, then read it back
- ▶ Easy: wrap a Week 2 age-input program in try/except so it does not crash on bad input
- ▶ Medium: save a list of 5 favorite foods to a file, one per line
- ▶ Medium: read a file of numbers and print only the ones above 50
- ▶ Challenge: build a simple logbook that appends (mode "a") a new line each run, without erasing previous entries

Complete Student Course Material

Phase 3: Python for Data

WEEK 6

6. Meet Your Data

WEEKLY GOAL

Understand what a dataset is, and load and look at a real one using Pandas.

CAREER CORNER · ON THE JOB AS AN AI/ML ENGINEER

This is one of the most important weeks in your journey toward becoming an AI or Machine Learning engineer. Ask any working engineer what they spend most of their time doing, and most will say working with data, not writing clever algorithms. Understanding rows, columns, features, and targets is not a small preliminary step, for most engineers it genuinely is most of the real work.

Key Words

Dataset, row, column, feature, label/target, structured data, CSV, DataFrame.

Concept

A dataset is simply a table of rows and columns, like a spreadsheet. Each row usually represents one example (one student, one house, one flower). Each column represents one piece of information about that example, called a feature, except for one special column we are trying to predict, called the label or target.

name	study_hours	attendance	passed
Amaka	5	90	yes
Tunde	1	60	no
Zainab	4	95	yes

Here, study_hours and attendance are features. passed is the target, what we would want a model to predict for a new student we do not already know the answer for.

DID YOU KNOW?

Some datasets used by professional AI/ML engineers contain billions of rows, far too large to fit in memory at once. Yet the underlying idea never changes: rows and columns, features and a target, exactly like the tiny table above. The tools change with scale, the core idea does not.

```
1 import pandas as pd
```

Complete Student Course Material

(Install once via Thonny's Manage Packages, as in the Setup Guide.) `pd` is just a short nickname for the library.

```
1 import pandas as pd
2 data = pd.read_csv("students.csv")
3 print(data)
4 print(data.head()) # first 5 rows only
5 print(data.columns) # column names
6 print(data.shape) # (rows, columns)
```

Guided Example

Load a small ready-made `students.csv` (5 to 10 rows: name, study_hours, attendance, passed). Print `.head()`, `.shape`, and `.columns`, and discuss what each row and column represents.

Try It Yourself

Using the same file, print just one column using `data["study_hours"]`, and print the number of rows using `len(data)`.

Hint. Selecting a column uses square brackets with the column name in quotes, just like a dictionary key.

```
1 data = pd.read_csv("student.csv")
```

DEBUGGING CHALLENGE

This crashes with `FileNotFoundError` even though `students.csv` exists. What is wrong? Hint: read the filename carefully, letter by letter.

MINI-PROJECT

Load `students.csv`, print how many rows and columns it has, print the column names, print the first 3 rows, and write, in a comment, one sentence describing what each column means.

AI / ML CONNECTION

This is the literal first step of nearly every real machine-learning project: get the data into your program and understand its shape before doing anything else. Skipping this step is one of the most common mistakes even professional data scientists make.

Practice Tasks

- Easy: load the CSV and print only its column names

Complete Student Course Material

- ▶ Easy: print the last 3 rows using `data.tail(3)`
- ▶ Medium: print just the attendance column
- ▶ Medium: use `data["passed"].value_counts()` and explain in a comment what it shows
- ▶ Challenge: create your own tiny CSV by hand describing 5 favorite foods with columns like name, price, is_healthy, then load it with Pandas

Complete Student Course Material

WEEK 7

7. Exploring Data

WEEKLY GOAL

Calculate simple statistics about a dataset and draw a basic chart from it.

CAREER CORNER · ON THE JOB AS AN AI/ML ENGINEER

Before training a single model, professional AI and Machine Learning engineers explore their data first: checking averages, hunting for mistakes, and drawing simple charts. Skipping this step is a common mistake even experienced adults make when in a hurry, and it usually costs far more time later.

Key Words

NumPy array, mean, Matplotlib, bar chart, line chart.

Concept

```
1 import numpy as np
2 scores = np.array([70, 85, 92, 61, 78])
3 print(scores.mean()) # average
4 print(scores.max())
5 print(scores.min())
```

NumPy is a library built for fast maths on lists of numbers, Pandas actually uses NumPy behind the scenes.

```
1 import pandas as pd
2 data = pd.read_csv("students.csv")
3 print(data["study_hours"].mean())
4 print(data["attendance"].max())
5 print(data.describe()) # summary of every numeric column at once
```

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 data = pd.read_csv("students.csv")
5 plt.bar(data["name"], data["study_hours"])
6 plt.xlabel("Student")
7 plt.ylabel("Study Hours")
8 plt.title("Study Hours per Student")
9 plt.show()
```

Guided Example

Complete Student Course Material

Using `students.csv`, calculate the average attendance, then plot a bar chart of attendance per student, and discuss what patterns are visible.

Try It Yourself

Plot a bar chart of `study_hours` per student, and add a comment stating which student studied the most and least, based only on the chart.

Hint. You need `plt.bar(x_values, y_values)`, then `plt.show()` at the end, do not forget it or nothing will appear.

```
1 | plt.bar(data["name"], data["study_hours"])
```

DEBUGGING CHALLENGE

Nothing appears when this runs. What is missing? Hint: the last line of the guided example.

MINI-PROJECT

Load `students.csv`, print `data.describe()`, then produce two charts: study hours per student, and attendance per student. Write two comments describing what each chart shows.

AI / ML CONNECTION

Exploring data before building a model is not optional, it is how real practitioners catch mistakes, spot patterns, and decide which features are worth using. The charts and averages you just produced are exactly what a data scientist does in the first hour of a real project.

Practice Tasks

- ▶ Easy: print the mean and max of the attendance column
- ▶ Easy: print `data.describe()` and identify the column with the highest average
- ▶ Medium: plot a bar chart of passed counts using `data["passed"].value_counts().plot(kind="bar")`
- ▶ Medium: create a NumPy array of 6 numbers and print its mean, max, and min
- ▶ Challenge: plot `study_hours` versus attendance as a scatter plot using `plt.scatter()`, and describe whether there seems to be a relationship

Complete Student Course Material

Phase 4: Introduction to AI and Machine Learning

WEEK 8

8. What Is AI and Machine Learning, Really?

WEEKLY GOAL

Build an accurate, non-magical mental model of what AI and ML actually are. No new syntax, this is a concept-first week.

CAREER CORNER · ON THE JOB AS AN AI/ML ENGINEER

The vocabulary you learn this week, model, training, features, labels, accuracy, overfitting, is not classroom-only language. It is the actual working vocabulary used in real AI and Machine Learning job interviews, research papers, and everyday conversation between engineers.

Key Words

Artificial Intelligence (AI), Machine Learning (ML), model, training, prediction, feature, target/label, training data, testing data, classification, regression, accuracy, overfitting.

Concept

Traditional programming, everything you have done Weeks 1 to 7:

Rules + Data → Answer

You wrote the rules (if score $\geq$ 50: print("Pass")). The computer just applies them. Machine Learning flips this around:

Examples + Answers → Model

Instead of a human writing the rules, we give the computer many examples with their known answers, and the computer works out the rules, the model, itself. Then, once trained:

New Data + Model → Prediction

We give the trained model a new example it has never seen, and it uses what it learned to predict an answer.

DID YOU KNOW?

The term "machine learning" was first used in 1959, by computer scientist Arthur Samuel, who taught a computer to get better at checkers by having it play many games and learn from the results. The core idea you learn this week, examples plus answers produce a model, is more than sixty years old. What has changed since is mostly the amount of data and computing power available, not the basic idea.

Concept

Complete Student Course Material

- ▶ Model: a program the computer builds itself, by finding patterns in examples, rather than a human writing every rule
- ▶ Training: showing the model many examples so it can find patterns
- ▶ Feature: an input the model uses to make its decision (e.g. study_hours, attendance)
- ▶ Target/Label: the answer we want the model to predict (e.g. passed)
- ▶ Training data: the examples used to teach the model
- ▶ Testing data: a separate set of examples, not used in training, used afterward to check how well the model actually learned
- ▶ Classification: predicting a category (e.g. "pass" or "fail")
- ▶ Regression: predicting a number (e.g. a price, a score)
- ▶ Accuracy: the fraction of predictions on testing data that were correct
- ▶ Overfitting: when a model memorizes its training examples instead of learning a real pattern, doing well on data it has seen before but poorly on new data

Complete Student Course Material

Unplugged Activity, "Human as a Model": on paper, give students 8 example rows of students.csv-style data as "training data," but hide the passed column on 2 new rows ("testing data"). Ask students to guess the missing answers using only the patterns they notice. Discuss: what pattern did you use? Did everyone use the same one? How confident are you? This is, in miniature, exactly what a model does.

Guided Demo: show students.csv again and discuss: "If I asked a computer to look for a pattern connecting study_hours and attendance to passed, without writing an if statement, what would it notice?" Connect this back to Week 6 to 7's exploration.

Check Yourself

1. In your own words, what is the difference between traditional programming and machine learning?
2. What is the difference between a feature and a target?
3. Why do we test a model on data it has not seen before, instead of the training data?
4. What might it mean if a model gets 100% accuracy on training data but very low accuracy on testing data?

AI / ML CONNECTION

This entire lesson is the AI/ML connection, every remaining lesson builds directly on this vocabulary.

Practice Tasks

- ▶ Easy: in your own words, explain what "training" means
- ▶ Easy: give one example of a classification problem and one of a regression problem
- ▶ Medium: explain why accuracy alone might be misleading with 95 "no" answers and only 5 "yes" answers
- ▶ Medium: explain overfitting using an example from school life, not exams
- ▶ Challenge: sketch what features you think would help predict whether a football team wins a match

Complete Student Course Material

Phase 5: Building AI/ML Projects

WEEK 9

9. Your First Model

WEEKLY GOAL

Train, test, and use a real machine-learning model using scikit-learn, with heavy guidance.

CAREER CORNER · ON THE JOB AS AN AI/ML ENGINEER

What you are about to do this week, load data, split it, train a model, test it, and use it to make a prediction, is the exact daily workflow followed by professional AI and Machine Learning engineers, at a much smaller and gentler scale. Professionals use larger datasets and more complex models, but these six steps almost never change.

Key Words

scikit-learn, train/test split, .fit(), .predict(), .score().

DID YOU KNOW?

The scikit-learn library you use this week is not a simplified "student version." It is the same real, professional library used inside many companies for genuine classification and regression tasks: spam filters, recommendation systems, fraud detection, and more. You are learning the real tool, on a smaller, friendlier dataset.

```
1 | pip install scikit-learn
```

(Or via Thonny's Manage Packages, searching scikit-learn.)

Complete Student Course Material

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.tree import DecisionTreeClassifier
4
5 # 1. Load the data
6 data = pd.read_csv("students.csv")
7
8 # 2. Choose features (X) and target (y)
9 X = data[["study_hours", "attendance"]]
10 y = data["passed"]
11
12 # 3. Split into training and testing data
13 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
14 random_state=42)
15
16 # 4. Choose and train a model
17 model = DecisionTreeClassifier()
18 model.fit(X_train, y_train)
19
20 # 5. Test the model
21 accuracy = model.score(X_test, y_test)
22 print("Accuracy:", accuracy)
23
24 # 6. Make a new prediction
25 new_student = [[3, 80]] # study_hours=3, attendance=80
26 prediction = model.predict(new_student)
27 print("Prediction:", prediction)
```

Explain each line as a continuation of Week 8's vocabulary: X is the features, y is the target, train_test_split creates separate training and testing data, .fit() is training, .score() is testing/accuracy, .predict() makes a prediction on new, unseen data.

Complete Student Course Material

Guided Example

Walk through the full block above together, live, using an expanded students.csv (at least 15 to 20 rows this week so the split makes sense).

Try It Yourself

Change test_size to 0.3 and re-run. Does accuracy change? Try changing new_student and predict again with different study_hours/attendance combinations.

Hint. You only need to change the numbers, not the structure of the code.

```
1 model = DecisionTreeClassifier()  
2 model.predict(X_test)
```

DEBUGGING CHALLENGE

This crashes. What critical step was skipped? Compare against the 6-step workflow above, one numbered step is entirely missing.

MINI-PROJECT

Using the expanded students.csv, build the complete 6-step pipeline from scratch, typed by hand, referring to the example only when stuck. Predict the outcome for 3 new made-up students and write one sentence per prediction explaining, in plain language, what the model predicted and why that seems reasonable.

AI / ML CONNECTION

This is the AI/ML connection, you just trained a real machine-learning model using the same library and core method (at a much smaller scale) that professional data scientists use.

Practice Tasks

- ▶ Easy: re-run the pipeline changing only random_state, does accuracy change?
- ▶ Easy: print X_train.shape and X_test.shape and explain what the numbers mean
- ▶ Medium: add a third feature to X if your CSV has one, and see if accuracy changes
- ▶ Medium: try RandomForestClassifier instead of DecisionTreeClassifier (same fit/predict/score pattern) and compare accuracy
- ▶ Challenge: predict outcomes for 5 new students with varied feature values, and write a short paragraph interpreting the pattern in the model's predictions

Complete Student Course Material

WEEK 10

10. Build With Less Help

WEEKLY GOAL

Apply the same 6-step ML workflow to a new problem, with reduced scaffolding, the bridge to full independence.

CAREER CORNER · ON THE JOB AS AN AI/ML ENGINEER

This week you practice something real AI and Machine Learning engineers do constantly: taking a workflow that worked for one problem and thoughtfully adapting it to a new one. This flexible, transferable thinking, rather than memorizing one fixed project, is what separates someone who can only follow a tutorial from someone who can actually build.

Key Words

Regression, LinearRegression, mean_absolute_error, feature/target selection (recap).

Concept

When the target is a number instead of a category, we use regression:

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import mean_absolute_error
5
6 data = pd.read_csv("students.csv")
7 X = data[["study_hours", "attendance"]]
8 y = data["exam_score"] # a number this time, not yes/no
9
10 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
11 random_state=42)
12
13 model = LinearRegression()
14 model.fit(X_train, y_train)
15
16 predictions = model.predict(X_test)
17 error = mean_absolute_error(y_test, predictions)
18 print("Average prediction error:", error)
19
20 new_prediction = model.predict([[4, 85]])
21 print("Predicted score:", new_prediction)
```

The workflow is the same 6 steps as Week 9, only the model type and how we measure success change. Accuracy does not make sense for numbers, so we use average error instead: "on average, our predictions were off by about this many points."

Complete Student Course Material

Guided-but-lighter walkthrough: instead of typing every line together, students are given the 6-step structure as headings only (Load, choose features/target, split, train, test, predict) and fill in each step themselves, checking the reference code only when stuck.

Try It Yourself

Using a provided offline dataset (a small fruit weight/size to fruit type set, or the classic small Iris flower dataset built into scikit-learn via `from sklearn.datasets import load_iris`), decide which columns should be features and which should be the target, then build the full pipeline.

Hint. Ask yourself "what am I trying to predict?", that column is your target, everything else useful becomes a feature.

```
1 | # split and training steps swapped in order
```

DEBUGGING CHALLENGE

Given a pipeline with `model.fit()` called before `train_test_split()`, find and explain the problem.

MINI-PROJECT

Choose from 2 to 3 small prepared datasets and independently complete the full 6-step pipeline, writing a short explanation of your feature/target choice before coding it.

AI / ML CONNECTION

This week proves the workflow from Week 9 was not a one-off: the same shape (load, choose features/target, split, train, test, predict) applies to a completely different problem and model type. That reusable shape is exactly what the final project will ask you to apply on your own.

Practice Tasks

- ▶ Easy: change the features used and see how the error changes
- ▶ Easy: predict the value for 3 new, self-invented inputs and sanity-check the numbers
- ▶ Medium: compare `LinearRegression` against `DecisionTreeRegressor` (same interface) on the same data
- ▶ Medium: write, in your own words, why we cannot use "accuracy" for a regression problem
- ▶ Challenge: pick a different pair of features from the same dataset and see whether the prediction error gets better or worse, and explain why

Complete Student Course Material

Phase 6: Independent Final Project

WEEK 11

11. Your Own Project

WEEKLY GOAL

Independently design, build, debug, test, and present a small machine-learning project from start to finish.

CAREER CORNER · ON THE JOB AS AN AI/ML ENGINEER

By completing this final project, you join a small and genuinely capable group of people: those who have taken a real problem, gathered data for it, trained a model on it, tested whether it worked, and explained the results clearly to someone else. That loop, problem, data, model, test, explain, is the loop a professional AI or Machine Learning engineer repeats their entire career, simply with bigger problems, bigger teams, and bigger datasets. You have now done it for real, yourself, from the ground up.

This week has no new syntax, every skill needed already exists from Weeks 1 to 10. This week is about applying everything you already know.

Session 1: Design and Build

Work through the Final Project Framework (Part 5) independently:

1. Choose a problem, from the provided list or your own idea, approved by your instructor
2. Identify what data is needed
3. Choose or create a small dataset
4. Load the data
5. Explore the data (shape, columns, simple stats or chart)
6. Choose features and target, and write one sentence justifying the choice
7. Split the data into training and testing sets
8. Train a model
9. Evaluate it (accuracy or error)
10. Make at least 2 new predictions and interpret them in plain language

Session 2: Improve, Test, and Present

1. Improve or experiment, try a different feature set, model, or more data, and compare results
2. Present the project to another student or the class: what problem it solves, what data it uses, how the model performs, and what a new prediction means, without reading from a script

Complete Student Course Material

Debugging and Support: keep a short "Bug Log", every error encountered, what it meant, and how it was fixed, reinforcing Week 1's error-first mindset one last time, now applied to your own original work.

AI / ML CONNECTION

This is the connection: the entire course was designed so that, by this point, you have already independently practiced every single piece this final project requires.

Exercises and Solutions

PART 4

Exercises and Solutions

Exercises for each week are listed in Part 3 under "Try It Yourself," "Mini Challenge," and "Practice Tasks." Solutions are given here, separated from the exercises, so students attempt each task before checking. Use the progression: task, think, hint (given in Part 3), try again, then the solution here.

Week 1 Solutions

Debugging Challenge

Missing closing parenthesis. Corrected:

```
1 print("Hello, World!")
```

Mini Challenge

```
1 # My About Me program
2 print("My name is Musa")
3 print("I am 13 years old")
4 print("My favorite subject is Basic Science")
5 print("I am learning Python this year")
```

Week 2 Solutions

Debugging Challenge

age is a string from input(), adding 1 (an int) to it fails. Fix:

```
1 age = int(input("Enter your age: "))
2 next_year = age + 1
3 print(next_year)
```

Missing colon after "if score >= 50". Fix:

```
1 score = 55
2 if score >= 50:
3     print("Pass")
4 else:
5     print("Fail")
```

Mini Challenge

Exercises and Solutions

```
1 name = input("Enter your name: ")
2 s1 = int(input("Enter score 1: "))
3 s2 = int(input("Enter score 2: "))
4 s3 = int(input("Enter score 3: "))
5 average = (s1 + s2 + s3) / 3
6 if average >= 50:
7     print(name, "passed with an average of", average)
8 else:
9     print(name, "did not pass. Average was", average)
```

Exercises and Solutions

Week 3 Solutions

Missing colon after "for score in scores". Try It Yourself (above-average scores):

```
1 scores = [70, 85, 92, 61, 78]
2 average = sum(scores) / len(scores)
3 for score in scores:
4     if score > average:
5         print(score)
```

Week 4 Solutions

Missing colon after "def add(a, b)". Try It Yourself (is_passing):

```
1 def is_passing(score):
2     return score >= 50
3
4 print(is_passing(72))
5 print(is_passing(40))
6 print(is_passing(50))
```

Week 5 Solutions

Try It Yourself

```
1 file = open("scores.txt", "r")
2 total = 0
3 for line in file:
4     total = total + int(line.strip())
5 file.close()
6 print("Total:", total)
```

Weeks 6 to 10 Solutions

Because these weeks work with datasets that vary by class, model solutions are given as fully worked code inside the "Guided Example" and "Mini-Project" sections of Part 3 for each week. The pattern shown there is the solution pattern, students substitute their own column and file names.

Week 11

There is no fixed "solution" for the final project by design, evaluation uses the rubric in Part 6.

Projects

PART 5

Projects

Weekly Mini-Projects

Full details for each are in Part 3, under each week's Mini-Project or Mini Challenge section.

1. Week 1, About Me program
2. Week 2, Pass or Fail grader
3. Week 3, Class Score Tracker
4. Week 4, Student Record System
5. Week 5, Save and Load Scores
6. Week 6, Dataset exploration report
7. Week 7, Two-chart data exploration
8. Week 8, "Human as a Model" reflection
9. Week 9, Guided Pass/Fail Predictor
10. Week 10, Choose Your Own Guided Project

Final Project Framework

For all final project options: 1. Choose a problem. 2. Identify the data needed. 3. Choose or create a dataset. 4. Load the data. 5. Explore the data. 6. Choose features and target. 7. Split the data. 8. Train the model. 9. Evaluate it. 10. Make predictions. 11. Improve or experiment. 12. Present the project.

Final Project Options

Choose one, or propose your own using the same framework.

Option A: Will I Pass? (Classification)

- ▶ Predict whether a student passes or fails based on study-related habits
- ▶ Dataset: a small class-collected CSV (study_hours, attendance, homework_done, passed), 20 rows is enough
- ▶ Features: study_hours, attendance, homework_done. Target: passed
- ▶ Model: DecisionTreeClassifier. Difficulty: beginner-friendly, the recommended default
- ▶ Extension: add more features such as sleep hours, or compare two classifier types

Option B: What Type of Flower Is This? (Classification, classic dataset)

- ▶ Predict a flower's species from its measurements
- ▶ Dataset: the built-in Iris dataset (from sklearn.datasets import load_iris), no internet or file needed

Projects

- ▶ Features: petal and sepal length and width. Target: flower species
- ▶ Model: classification. Difficulty: beginner-friendly
- ▶ Extension: visualize with a scatter plot first, or predict using only 2 of the 4 features

Projects

Option C: Predicting Exam Scores (Regression)

- ▶ Predict a numeric exam score from study habits
- ▶ Dataset: the same class CSV as Option A, with an added exam_score column
- ▶ Features: study_hours, attendance. Target: exam_score (a number)
- ▶ Model: LinearRegression. Difficulty: beginner to intermediate
- ▶ Extension: compare Linear Regression against a Decision Tree Regressor

Option D: Fruit or Vegetable Classifier (very small dataset)

- ▶ Classify an item as fruit or vegetable from simple measurable properties
- ▶ Dataset: a tiny, hand-typed CSV of about 15 rows (weight_grams, is_sweet, has_seeds_outside, type), typeable by hand in 10 minutes with no download needed
- ▶ Model: classification. Difficulty: beginner-friendly, recommended for very limited-resource situations
- ▶ Extension: add more rows collected as a class, or add a third category

Option E: Simple Movie/Book Recommendation

- ▶ Suggest whether someone would enjoy a book or movie based on genre-preference patterns
- ▶ Dataset: a small hand-built CSV of about 20 rows (genre, length, liked) from real class preferences
- ▶ Features: genre (converted to numbers, a light preview of encoding categories) and length. Target: liked
- ▶ Model: classification. Difficulty: intermediate, introduces encoding text as numbers
- ▶ Extension: build it from the class's own real survey data

Assessments

PART 6

Assessments

Weekly Exercises and Quizzes

Each week's "Check Yourself" and "Exit Ticket" sections (Part 3) work as short formative quizzes (3 to 5 questions), used at the start of the next session as a review warm-up.

Sample Mini-Quiz Format

Used at the start of sessions, 5 minutes. Example, Week 3 Session 2 warm-up:

1. What does `scores[0]` return in a list `scores = [10, 20, 30]`?
2. What is the difference between `for` and `while`?
3. What would happen if a `while` loop's condition never becomes `False`?

Coding Challenges

The "Challenge" item in each week's Practice Tasks (Part 3) serves as that week's coding challenge, gradually increasing in required independence.

Mid-Course Practical Assessment (End of Week 6)

A 45-minute, closed-notes-but-open-own-code practical:

1. Write a program using a list, a loop, and an if statement to process 5 typed scores and print who passed or failed. (40%)
2. Load a provided small CSV with Pandas and print its shape, columns, and the average of one numeric column. (40%)
3. Explain, in 2 to 3 sentences, what a "feature" and a "target" mean, using an example from the CSV. (20%)

Final Project Assessment

Combines the completed project (code plus a short write-up) and a short live or recorded explanation (2 to 4 minutes) of the project.

Final-Project Rubric (100 points total)

Criterion	Weight	What "Strong" Looks Like
Problem understanding	15	Can clearly state, in their own words, what problem the project solves and why it matters
Python code quality	15	Code runs, is reasonably organized, uses appropriate variables, functions, and loops

Assessments

Criterion	Weight	What "Strong" Looks Like
Data handling	15	Data is loaded correctly, can describe what each feature and the target represent
ML implementation	20	Correct use of the 6-step workflow: split, train, test, predict
Testing/evaluation	10	Reports and correctly interprets accuracy or error
Experimentation	10	Made at least one deliberate change (feature, model, data) and compared results
Explanation	10	Can explain the project to someone else, unscripted, in plain language
Independence	5	Project reflects the student's own decisions, not a copied tutorial

Presentation polish (slides, visuals) is not separately scored. It may support the "Explanation" criterion only if it aids clarity, never as a criterion on its own, following the course's technical-understanding-first principle.

Final Project Readiness Matrix

PART 7

Final Project Readiness Matrix

Skill	Introduced	Practiced	Final Project Use
Variables	Week 1	Weeks 1 to 11	Storing inputs, results, and predictions
Conditions (if/elif/else)	Week 2	Weeks 2, 3, 9 to 11	Interpreting predictions, branching logic
Lists	Week 3	Weeks 3, 4, 6 to 11	Holding features, scores, results
Loops	Week 3	Weeks 3 to 11	Iterating through data rows
Functions	Week 4	Weeks 4, 9 to 11	Reusable prediction/interpretation helpers
Dictionaries	Week 4	Weeks 4, 6	Understanding row/record structure before Pandas
Files (read/write)	Week 5	Weeks 5, 6	Loading and saving datasets
Error handling (try/except)	Week 5	Weeks 5, 9 to 11	Handling bad input or missing files gracefully
Datasets, rows, columns	Week 6	Weeks 6 to 11	Understanding the project's data
Pandas	Week 6	Weeks 6, 7, 9 to 11	Loading and inspecting the dataset
Data visualization	Week 7	Weeks 7, 11	Exploring data before modeling
Features/target	Weeks 6 and 8	Weeks 8, 9, 10, 11	Selecting X and y
Train/test split	Week 9	Weeks 9, 10, 11	Splitting the project's dataset
ML model (fit)	Week 9	Weeks 9, 10, 11	Training the project's model
Evaluation (score/error)	Week 9	Weeks 9, 10, 11	Reporting how good the model is
Prediction	Week 9	Weeks 9, 10, 11	Producing and interpreting new predictions

Final Project Readiness Matrix

READINESS CHECK

Every row above shows the skill introduced at least one full week before Week 9, the first week it is required for real model-building, and practiced at least once more before Week 11. No skill required by the final project is introduced for the first time during Weeks 10 or 11. A student who has completed Weeks 1 to 10 in full has already personally practiced, at least once, every single skill the final project requires.

Closing Note From Mr. Muwatta

If you have followed this material week by week, from your very first `print("Hello, World!")` to your own finished Machine Learning project, you have done something many people twice your age have never done. Be proud of that.

Keep exploring, keep experimenting, and keep asking "why" whenever your code does not do what you expected. That question is the whole of programming.

Mr. Muwatta

Founder, Algorise Tech Explorers
www.muwatta.com.ng